

Aula 06 - Várias contas com arrays e índice

Contexto de negócio

Processamento em lote: manipular várias contas com arrays e índices consistentes.

Uma única conta já não representa o sistema. Como modelar várias contas no procedural?

Tensão operacional

Com variável única de saldo, não conseguimos separar contas diferentes.

Observação didática: arrays paralelos funcionam, mas esta fragilidade é intencional na trilha. Se as posições entre arrays se desalinharem, o sistema mistura dados. Esse problema será analisado com profundidade nas próximas etapas da trilha.

Objetivo técnico da entrega

- Arrays paralelos.
- Índice como identificador operacional.
- Evidenciar, com demonstração, o que mudou em relação ao cenário de uma conta.

Recurso técnico desta aula

Recurso inserido: arrays de titulares e saldos, com métodos que recebem `array + índice`.

Problema que justifica o novo artefato

Uma única variável de saldo não representa operação real com múltiplas contas. Sem estrutura para volume, não há algoritmo de lote consistente.

Artefato explicado: arrays com operações por índice

Artefatos desta aula:

- arrays para titulares e saldos;
- métodos que recebem `array + índice`.

Cada índice identifica uma conta operacional no algoritmo. Isso permite repetição de regra sem duplicar função por cliente.

O que mudou de fato (antes e depois)

Pergunta que o aluno faz com razão: "o que mudou, na prática?"

Antes (uma conta):

- uma variável de saldo;
- uma chamada de saque;
- risco menor de confusão estrutural.

Depois (várias contas):

- estrutura para coleção de saldos e titulares;
- operações direcionadas por índice;
- novo risco: o algoritmo pode operar a conta errada se o índice ou o alinhamento estiver incorreto.

Comparação direta:

```
// Antes (uma conta)
double saldo = 1000.00;
saldo = saldo + 100.00;

// Depois (várias contas)
double[] saldos = {1000.00, 700.00};
saldos[0] = saldos[0] + 100.00;
```

Valor agregado real da aula:

- deixa explícito o custo técnico de sair de uma conta para N contas;
- prepara o aluno para reconhecer erros estruturais, não só erros de sintaxe.

Demonstração prática de impacto

Cenário A (alinhado):

- `titulares[0] = "Ana"` e `saldos[0] = 1000.00`;
- depósito no índice `0` aumenta o saldo da Ana.

Cenário B (desalinhado):

- troca apenas `titulares` de ordem;
- depósito continua no índice `0`, mas agora afeta outro titular na leitura.

Conclusão operacional:

- o código compila e executa;
- o resultado de negócio pode ficar errado sem lançar erro.

Esse é o tipo de falha que times experientes mais combatem.

Variação procedural: "array de contas" sem mudar de paradigma

É possível tentar uma representação mais "agrupada" ainda no procedural, por exemplo:

```
String[] titulares = {"Ana", "Bruno"};
double[][] contaNumerica = {
    {1000.00, 500.00}, // saldo, limite da conta 0
    {700.00, 200.00}  // saldo, limite da conta 1
}
```

```
};  
boolean[] ativas = {true, true};
```

O que melhora:

- parte numérica da conta fica agrupada em uma matriz.

O que continua limitado:

- tipos continuam espalhados em estruturas diferentes;
- dependência de convenção de posição (ex.: coluna 0 = saldo, coluna 1 = limite);
- manutenção ainda frágil quando o modelo cresce.

Pergunta direta desta aula: "dá para ter a conta toda em um único elemento de array?"

Resposta objetiva:

- com array tipado normal em Java, não dá para misturar `String`, `double` e `boolean` no mesmo elemento sem abrir mão de tipagem forte;
- arrays em Java são homogêneos por tipo.

Então, no procedural puro, as opções reais são:

1. Separar por tipo (modelo atual): mantém tipagem, mas espalha dados.
2. Usar `Object[][]`: permite juntar tudo por linha, mas exige cast e aumenta risco de erro.
3. Colocar tudo em `String[][]` e converter/validar em cada operação: funciona, mas adiciona complexidade operacional e risco de parse.

Exemplo de linha única com `Object[][]`:

```
Object[][] contas = {  
    {"Ana", 1000.00, 500.00, true},  
    {"Bruno", 700.00, 200.00, true}  
};
```

Limites dessa opção:

- precisa de cast para ler valores;
- erros podem aparecer só em tempo de execução;
- clareza do algoritmo cai para quem está começando.

Pergunta guia para a turma:

- em que momento essa convenção deixa de ser sustentável para o time?
- qual custo de manutenção é maior: dados separados por tipo ou dados agrupados sem tipagem forte?

Transição didática para a próxima aula

O ganho de volume expõe um limite: dados e comportamento ainda estão separados. A próxima aula aprofunda esse limite em novo nível de organização de código.

Valor prático entregue

- Aumenta confiabilidade de operação em massa.
- A base fica pronta para evolução controlada da próxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.
- Executar a regra principal e observar o impacto no estado.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

Código em foco

```
// Classe temporaria de fluxo do programa.
// Neste curso, ela existe para iniciar a execução em Java.
public class AppGestaoContas {

    static void depositar(double[] p_saldos, int p_indice, double p_valor) {
        p_saldos[p_indice] = p_saldos[p_indice] + p_valor;
    }

    static boolean sacar(double[] p_saldos, int p_indice, double p_valor) {
        if (p_valor > 0 && p_valor <= p_saldos[p_indice]) {
            p_saldos[p_indice] = p_saldos[p_indice] - p_valor;
            return true;
        }
        return false;
    }

    static void mostrarContas(String[] p_titulares, double[] p_saldos) {
        for (int i = 0; i < p_saldos.length; i++) {
            System.out.println(p_titulares[i] + " | Saldo: " + p_saldos[i]);
        }
    }

    public static void main(String[] args) {
        // Aula 06: várias contas por índice.
        String[] titulares = {"Ana", "Bruno"};
        double[] saldos = {1000.00, 700.00};

        depositar(saldos, 0, 100.00);
        sacar(saldos, 1, 50.00);

        mostrarContas(titulares, saldos);
    }
}
```

Leitura técnica orientada

Percorra o código com estes checkpoints de revisão:

- Regra de decisão: mapear condições que aprovam ou negam operações.
- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

Discussão de contrato de retorno

Quando o método retorna `boolean`, o contrato precisa ser tratado como regra de negócio:

- `true` significa operação aprovada e estado atualizado;
- `false` significa operação negada e estado preservado.

Risco de lógica silenciosa:

- ignorar o `false` no chamador cria fluxo aparentemente correto, mas operacionalmente incompleto.

Responsabilidade de quem chama:

- sempre tratar os dois ramos (`true` e `false`) com mensagens e ações coerentes;
- registrar motivo de negativa quando possível (conta inativa, valor inválido, limite excedido);
- evitar seguir o fluxo como se a operação tivesse ocorrido.

Variações para debate técnico:

1. Em quais cenários `boolean` basta?
2. Quando o time precisa de retorno com motivo da falha?
3. Qual custo de não tratar o `false` em produção?

Paradigma em foco: imperativo, procedural e algoritmo

Leitura de paradigma nesta etapa:

- Imperativo: o programa descreve passo a passo o que a máquina deve executar.
- Procedural: organizamos esse passo a passo em funções reutilizáveis.
- Algoritmo: sequência finita de entrada, processamento, decisão e saída para resolver um problema real.

Por que passar valores nas funções no procedural:

- A função não “conhece” a conta por contexto implícito.
- Tudo que a regra precisa deve chegar por parâmetro.
- O contrato da função fica explícito, testável e auditável.

Risco comum quando isso não fica claro:

- chamar função com parâmetro errado, em ordem errada ou incompleto;
- gerar resultado válido na sintaxe, mas incorreto no negócio.

Ponte para a próxima virada de paradigma:

- No procedural, o estado circula entre funções.
- Em abordagens posteriores, a modelagem tende a reduzir o espalhamento estrutural dos dados.

Perguntas de decisão

1. O que o índice representa nesta aula?
2. Qual risco dos arrays paralelos?
3. O que mudou de forma concreta em relação à aula anterior?
4. Em qual ponto o desalinhamento passa a ser risco inaceitável?
5. A variação com matriz numérica reduz mesmo a fragilidade ou só desloca o problema?
6. Entre `Object[][]`, `String[][]` e arrays paralelos, qual opção gera menos risco para o time neste momento?

Variações de cenário

1. Trocar ordem dos titulares e observar confusão.
2. Adicionar terceira conta e operar nela.
3. Criar `contaNumerica` e comparar legibilidade com arrays paralelos.

Exercícios progressivos

1. Criar array `numerosConta` e imprimir junto.
2. Implementar saque para a terceira conta.
3. Montar um teste manual com arrays desalinhados e registrar o erro de negócio observado.

Desafio de equipe

Criar método `transferir(double[] saldos, int origem, int destino, double valor)`.

Critério extra do desafio:

- demonstrar em 5 linhas por que o método é sensível a índice incorreto.

Critérios de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.
- Código permanece legível e coerente com o nível da aula.

Checklist de progressividade técnica

- Pré-requisito confirmado: leitura e execução do código-base da aula anterior.
- Novidade técnica identificada: explicar em uma frase o recurso novo desta aula.
- Complexidade controlada: apontar qual decisão de projeto evita acoplamento desnecessário.
- Evidência prática: executar ao menos um caso de sucesso e um caso de erro no Tryit.

Fechamento executivo

Arrays resolvem volume inicial, mas aumentam risco de desalinhamento. Na próxima, vamos elevar o nível de organização para reduzir fragilidade estrutural.

Revisão rápida (5 minutos)

1. Regra central: qual condição decide aprovação ou bloqueio na aula?
2. O que mudou objetivamente em relação à versão de uma conta?

3. Qual foi o principal risco novo introduzido por índices?
4. Evolução: qual pequena extensão agregaria mais valor sem aumentar acoplamento?

Mini-missão de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

Execução no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler
2. Colar todo o conteúdo de AppGestaoContas.java no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercícios.
4. Manter uma aba separada para cada exercício e comparar resultados.

Referências W3Schools da aula

- https://www.w3schools.com/java/java_arrays.asp
- https://www.w3schools.com/java/java_methods.asp
- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>